8 Experimental Evaluation

This section tests the four hypotheses of the evaluation plan (H1–H4) and the composition and evolution questions (RQ3, RQ4) on the registries of Table 1. Results are reported as a service consumer sees them: how many complementary services the agent receives, what they cost in context, whether the tools a task needs are among them, which unsafe services become reachable, and what the discovery layer costs to run and maintain. All results come from a reference implementation of the capability model, Algorithms 1–3 and the certificate. No MCP server was executed, so effects are judged against labelled ground truth rather than observed side effects (Section 8.10).

8.1 Experimental Setup

Registries and tasks. LiveMCPTool [1] (69 servers, 525 tools) and MCPToolBench++ [2] (10 servers, 87 tools) are merged by `server::name` into 76 servers and 569 tools. The test split is the 95 LiveMCPBench tasks, whose annotated tool chains give 256 gold tool mentions; 29 resolve to several `server::name` identities, any of which counts, and five name tools absent from the registry. The public MCPToolBench++ release holds 1,509 single-call requests (1,470 distinct queries, against the 1,429 requests of Table 1). Ten queries per category, 60 in total, form the development split on which every parameter is tuned; all 1,509 requests are used where no analyst is involved.

Ground truth, evidence and adversary. The true effect relation J was labelled once per tool by an LLM (Claude Sonnet) from its name, description and schema, under a fixed guide that ignores annotations and safety claims: scoped effects, whether the tool modifies persistent, shared or external state, and whether its output can contain the principal's private data. Under J, 131 tools (23.0%) modify state and 80 can return private data. The same pass produced the LLM-assisted typing τ of Section 6.1 used for contract extraction; J never enters the engine as evidence. Observed evidence comes from a simulated sandbox verifier bound to $h(t)$, truthful except that it accepts a false read-only claim with a controlled false-accept rate (FAR); its coverage is $\kappa = 0.9$, and 30% of its records are signed by a trusted attester. As in Section 7, an attacker controls a fraction α of the servers, declares all their tools read-only and non-destructive and launders their descriptions, but cannot alter verifier evidence.

Model profile and LLM roles. ρ_{LLM} was measured for Claude Sonnet and Claude Haiku on 64 items: four classes (exact sum, exact count, majority sentiment, record lookup) $\times\ n \in \{10, 40, 160, 640\} \times$ four items, answered from the data in context without tools, with an isotonic fit and $\tau = 0.9$. Sonnet answered all 64 correctly, including exact sums of 640 numbers; Haiku's exact sum fell from 100% at $n = 40$ to 25% at $n = 160$ and 0% at $n = 640$. Rule 1 nevertheless fixes exact classes below τ beyond $n = 20$. Classes outside the calibration receive declared priors; treating them as unknown (strict rule 2) is an ablation. The analyst role was run once with Sonnet on all 158 prompts (three scenarios, 95 test, 60 development) and with Haiku on the 98 scenario and test prompts; the recorded goal contracts are replayed, so every run is deterministic.

TABLE 7
Experiments and the evidence they produce. Meas.: deterministic engine on registry data. LLM: recorded LLM role. Lab.: LLM-labelled ground truth J. Sim.: simulated verifier or attacker. Syn.: generated registries.

Exp.	Question	Conditions	Evidence
E0	H1 (registry)	569 tools, annotations, J	Meas., Lab.
E1	H1 (behaviour)	sandbox traces	*not run*
E2	H2	$\alpha \le 0.5$, FAR ≤ 0.2, 5 seeds	Meas., Lab., Sim.
E3	H3	95 test, 60 dev. tasks	Meas., LLM
E4	H4, Prop. 1	5 attack goals, 3 seeds	Meas., Lab., Sim.
E5	RQ3	48 hidden typed chains	Syn.
E6	Ablations	test split; 1,509 requests	Meas., LLM
E7	RQ4	10^3–10^5 tools, 600 events	Meas., Syn.
E8	Typing, Θ	525 tools, MCP-Bench format	Meas., LLM

Implementation and parameters. The engine is written in Python. Unless stated otherwise $K = 5$, $d = 2$, $N_{\max} = 40$, $u_{\min} = 0.5$, $w_0 = 1$ and $\beta = \gamma = \delta = \eta = \zeta = 0.5$ in Eq. (6), $k_0 = 60$ in Eq. (5), and the default trust policy assigns strengths 0.25 (lex, raised to 0.45–0.50 for delete, execute and payment cues), 0.50 (ann), $0.55 + 0.20\kappa$ (obs) and 1.0 (att, trusted signer) against thresholds $\ell_{\mathrm{lex}} = 0.25$, $\ell_{\mathrm{ann}} = 0.50$, $\ell_{\mathrm{obs}} = 0.65$ and $\ell_{\mathrm{att}} = 0.90$. Two components deviate from Section 6.3. The dense score uses a local latent-semantic encoder (TF-IDF over word and character n-grams, 128 dimensions) because BGE-M3 [3] could not be installed; and Eq. (6) gains a relevance term $w_{\mathrm{rel}}(1 - \hat{s}(e))$, with $\hat{s}(e)$ the candidate's fused retrieval score normalised per goal, where $w_{\mathrm{rel}} = 0$ recovers Eq. (6) and $w_{\mathrm{rel}} = 10$ was selected on the development split. The flow bound (L) is checked conservatively: every tool result enters the model's context and may reach any text-renderable argument of an egress tool. Runs used two virtual CPUs, 7 GB of memory and no GPU.

Baselines, metrics and statistics. The approaches of Table 2 are re-implemented over the same registry and retriever: RAG-MCP [4] (dense top-k over the whole prompt), MCP-Zero [5] (server routing, then one tool per goal), per-subtask retrieval (one tool per analyst goal) and the full catalogue. Metrics are toolkit size $|K|$, context tokens of the exposed definitions, gold-tool recall (the share of a task's gold tools in K), precision, abstention, exposure (the share of truly unsafe tools admitted), attack success and latency. Means carry 95% confidence intervals; paired comparisons over tasks use Wilcoxon signed-rank tests with bootstrap intervals for the mean difference; proportions use Wilson intervals. Table 7 lists the experiments and the kind of evidence each produces. Code, recorded LLM outputs and raw results are available.[1]

8.2 Registry Characterisation (H1, Registry Level)

The merged registry reproduces Table 1: 76 servers, 569 tools and 44 annotated tools, all from three servers and all claiming `readOnlyHint=true`; under J exactly one of them is not read-only. Requiring declared read-only hints keeps 7.7% of the registry, and reading a missing hint as safe removes nothing, because no tool declares

1. https://github.com/msellamiTN/taas-provenance-discovery

TABLE 8
Toolkit size, context cost and gold-tool recall (H3). Means over tasks; recall with its 95% confidence interval. LMB: LiveMCPBench, 95 tasks (test). TB++: MCPToolBench++, 60 queries (development). Tok.: tokens of the exposed tool definitions.

	LMB (test)			TB++ (dev)		
System	$\|K\|$	Tok.	Recall	$\|K\|$	Tok.	Recall
TaaS	1.42	126	$.186 \pm .054$	1.00	88	$.383 \pm .124$
no gap analysis	1.96	165	$.175 \pm .054$	0.90	84	$.367 \pm .123$
strict rule 2	1.92	162	$.182 \pm .056$	1.05	94	$.367 \pm .123$
Per-subtask	2.57	343	$.249 \pm .066$	1.32	252	$.433 \pm .126$
gap goals only	1.72	219	$.202 \pm .061$	1.05	236	$.417 \pm .126$
RAG-MCP top-3	3.00	745	$.240 \pm .065$	3.00	617	$.467 \pm .127$
RAG-MCP top-5	5.00	1,252	$.293 \pm .073$	5.00	1,089	$.550 \pm .127$
MCP-Zero	2.59	386	$.213 \pm .061$	1.28	220	$.317 \pm .119$
Full catalogue	569.00	96,420	1.000	569.00	96,420	1.000

`readOnlyHint=false`. The share of state-modifying tools is 23.0% under J and 32.2% under the lexical labeller of Section 6.1 (precision 0.60, recall 0.84 against J, 21 unsafe tools missed), bracketing the 27.8% of Section 2.2. Lexical evidence is both noisy and incomplete, which supports never admitting it for write-sensitive policies. H1 proper, a disagreement between declared and sandbox-observed behaviour, requires executed servers and is not tested here.

8.3 Provenance-Sound Eligibility (H2)

Fig. 8 compares six encodings of a read-only policy. MCP's default reading (ann+, admit declared read-only tools) admits 0.8% of the unsafe tools when every server is honest, 29.3% when a quarter forge their hints and 51.9% when half do. Reading a missing hint as safe (ann-cw) admits 85.5% even with honest servers, as Observation 1 predicts for a registry in which almost no tool is annotated, and the lexical encodings admit between 16.0% and 59.4%. Verified evidence (obs+) admits no unsafe tool at a perfect verifier for every α, as Proposition 2(a) states; its exposure is independent of α and grows with the verifier's error, to 1.5%, 5.8%, 11.5% and 22.0% at FAR of 1%, 5%, 10% and 20%.

Soundness costs utility through coverage, not through forgery. At $\alpha = 0.25$ and FAR 5%, obs+ keeps every safe tool because the verifier probes the whole registry, against 34.6% for ann+; when it probes only a quarter of the tools, exposure falls to 0.6% but only 24.4% of the safe tools remain eligible. The results also quantify the condition of Section 7 that a verifier helps only if its false-accept rate is below the error rate of the evidence it replaces. With honest servers the declared hints of this registry are almost all truthful, and a verifier at FAR 1% (1.5%) is already worse than ann+ (0.8%); once a quarter of the servers forge metadata, obs+ admits fewer unsafe tools than ann+ at every tested FAR. H2 is supported.

8.4 Model-Conditioned Gap Analysis (H3)

The analyst writes 2.96 goals per test task and the engine places 1.77 of them in the gap, so 40.2% of the goals are covered by the model; over the 89 certified toolkits the mean bridge ratio (B) is 0.12 and 70.8% have $\beta_K = 0$, because a native goal whose output type is already in the context is discharged by that type rather than by a model hyperedge. Table 8 compares the resulting toolkits with the baselines. Gap analysis removes 0.54 tools per task relative to the same engine without it (95% CI of the difference $[-0.68, -0.38]$, $p = 2.5 \cdot 10^{-8}$) with no detectable change in gold recall (0.186 vs. 0.175, $p = 0.62$). Against per-subtask retrieval TaaS exposes 1.15 fewer tools and 63% fewer tokens (126 vs. 343, $p < 10^{-12}$); its recall is lower (0.186 vs. 0.249), but not significantly (CI $[-0.144, 0.017]$, $p = 0.16$), and neither is the difference to MCP-Zero ($p = 0.56$). RAG-MCP top-5 reaches higher recall (0.293, $p = 0.02$) with ten times the context and less than half the precision (0.135 vs. 0.318). On the development split TaaS again exposes the fewest tokens, with recall 0.383 against 0.433 for per-subtask retrieval ($p = 0.41$) and 0.550 for RAG-MCP top-5 ($p = 0.0075$). TaaS abstains with a typed χ_1 counterexample on 6% of the test tasks, every other selection carries a valid certificate, and discovery takes 0.16 s per task.

Under Eq. (6) as stated, development recall falls as the retrieval budget grows (0.317, 0.200 and 0.100 for $K = 3$, 5 and 10; Fig. 9): with more type-compatible candidates, the minimum-cost cover selects the one with the fewest effects and strongest evidence however poorly it matches the goal. With $w_{\text{rel}} = 10$ recall stays between 0.350 and 0.383, and on the test split removing the term lowers recall from 0.186 to 0.082. The objective therefore needs a relevance component beside effect breadth, evidence and declared QoS. H3 is supported for toolkit size and context cost; gold-tool recall, a proxy for task success, is not significantly below per-subtask retrieval and is below RAG-MCP top-5.

8.5 Certified Toolkits under Prompt Injection (H4)

For each task the policy allows only the effect classes the analyst's goals require, with evidence at ℓ_{obs}. An injected instruction then requests an action in a class the task does not need: exfiltration (a private source together with an arbitrary-destination egress tool), deletion, code execution, payment, or any other state change. In the worst case the attack succeeds if the exposed toolkit can truly realise it under J, whatever the model is told after discovery. Fig. 10a shows that certified toolkits allow none of the 2,136 out-of-scope attacks at FAR 0 or 5% (Wilson upper bound 0.2%). Per-subtask retrieval allows 3.2% (95% CI 2.6–4.1%), MCP-Zero 3.5% and RAG-MCP top-5 8.1% (7.1–9.4%), mostly state changes (33% of its tamper attempts) and exfiltration (7%); the full catalogue allows all. The bound costs coverage: under these least-privilege policies the certified toolkits reach gold recall 0.188, against 0.253–0.393 for the retrieval baselines, because the policy also excludes tools the tasks need.

Property 1 was checked by executing random sequences of one to six calls over certified toolkits with the true effects of J. No sequence among 83,600 performed an effect outside A. Flows behave differently (Fig. 10b): if (L) propagates labels only along the derivations recorded by the composition, 16.6% of the sequences at FAR 0 (3,559 of 21,450) carry private data into an egress tool, because every result returns to the context and can be written into any text argument; with the conservative reading none does. Order independence for flows therefore holds under the conservative reading, which should accompany Property 1.

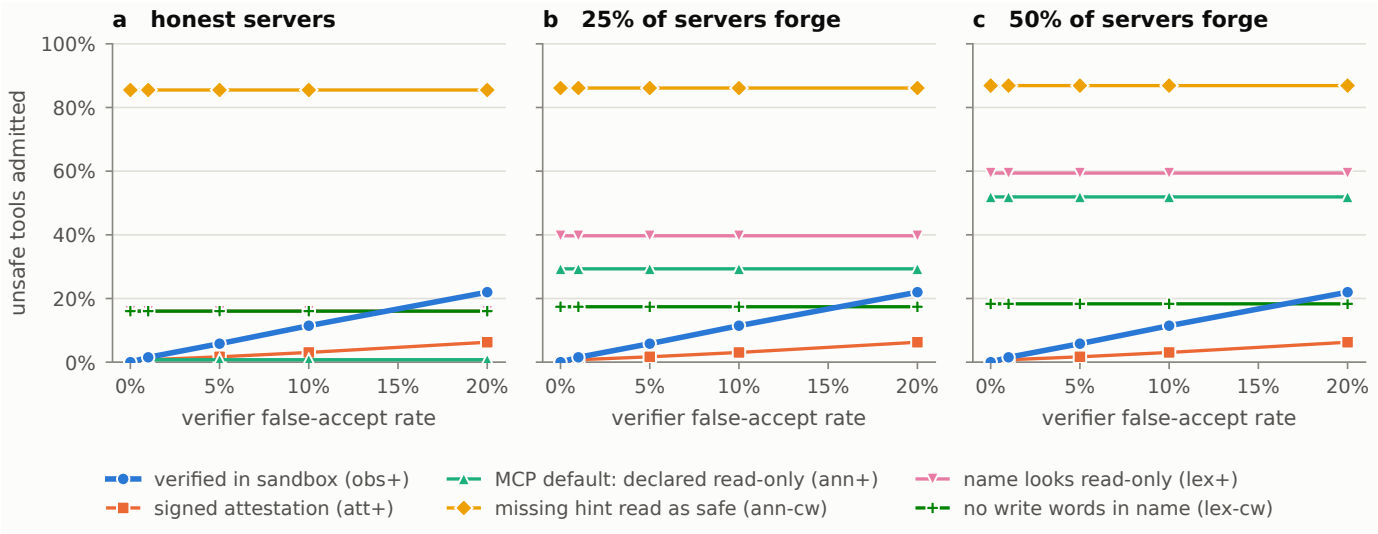


Fig. 8. Exposure under a read-only policy: share of truly unsafe tools (under J) admitted by each encoding, against the verifier's false-accept rate, when $\alpha = 0$, 0.25 and 0.5 of the servers forge metadata (569 tools, five seeds). Metadata-only encodings do not depend on the verifier.

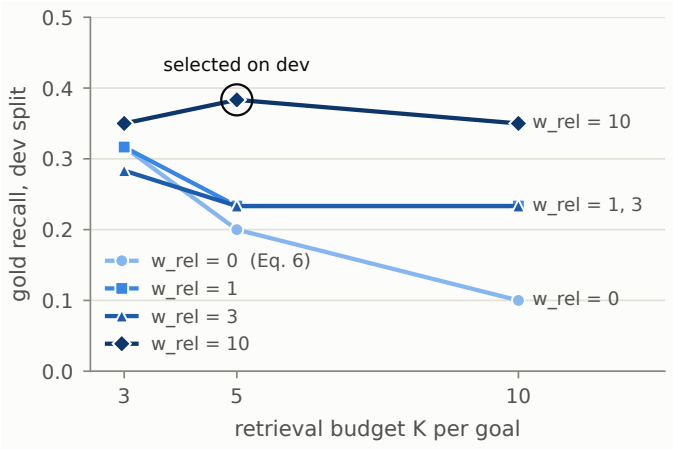


Fig. 9. Gold recall on the development split against the retrieval budget K and the weight w_{rel} of the relevance term; $w_{\mathrm{rel}} = 0$ is Eq. (6).

Under it the revenue scenario of Fig. 3 needs the χ_5 repair that splits the task into two toolkits; the split reproduces the toolkit of Fig. 3 and hands the aggregated revenue table over at label *internal*. The sanitised manifest, finally, removes description-borne injection: of 60 descriptions poisoned in the style of MCPTox [6], all reach the model in the raw tool list and none in the manifest, which also shortens definitions from 169 to 96 tokens per tool. H4 is supported; call-time controls [7], [8], [9] remain necessary for the arguments of permitted calls.

8.6 Composition on Controlled Registries (RQ3)

Real registries give no ground truth for optimality, so RQ3 uses generated registries. Each task hides a typed chain of length $L \in \{2, 3, 4\}$, and every step has three or five distractors: an unobtainable input type, lexical evidence only, a forbidden delete effect, a costlier twin, or a lexically closer tool of the wrong type. The goals name only one intermediate and the final type, so the other intermediates must be found by backward closure. Validity is graded by an independent checker and the optimum by brute force. Table 9 shows that both regimes return a valid, minimum-cost toolkit for all 48 tasks (Wilson lower bound 92.6%) in about 12 ms. Without backward closure 3 tasks are solved, and per-subtask retrieval solves all 32 tasks with $L \leq 3$ but none with $L = 4$: the closure over unnamed intermediate types is what makes multi-step toolkits complete.

TABLE 9
Composition on synthetic typed registries (RQ3): 48 tasks with hidden chains of length $L \in \{2, 3, 4\}$. Valid: toolkits accepted by an independent checker, with the 95% Wilson interval (%). By L: valid share (%) for $L = 2/3/4$. Opt.: share (%) whose cost equals the brute-force optimum.

Regime	Valid	95% CI	By L	Opt.	ms
Exact (B&B)	48/48	[92.6, 100.0]	100/100/100	100	12.1
Greedy cover	48/48	[92.6, 100.0]	100/100/100	100	11.5
No backward closure	3/48	[2.1, 16.8]	12/6/0	6	5.2
Per-subtask	32/48	[52.5, 78.3]	100/100/0	67	–

8.7 Ablations

Table 10 changes one component at a time. The relevance term has by far the largest effect. Reading Algorithm 1 literally, with only native outputs entering the context (line 15), demotes native goals downstream of data goals and adds 0.51 tools per task, and strict rule 2 adds 0.49; both cost context rather than coverage. Of the role ablations in the evaluation plan only the analyst's model was varied: Haiku instead of Sonnet lowers recall by 0.010 and adds 0.18 tools. Refinement ran with its deterministic operators only, without the assistant role; disabling it raises abstention from 6% to 8%. Backward closure matters little on LiveMCPBench (recall -0.004), unlike on the chains of Section 8.6. For retrieval alone, on all 1,509 MCPToolBench++ requests, BM25 reaches R@5 of 0.613, the latent-semantic encoder 0.543 and their reciprocal-rank fusion 0.588, below its stronger component because the dense stand-in is weak.

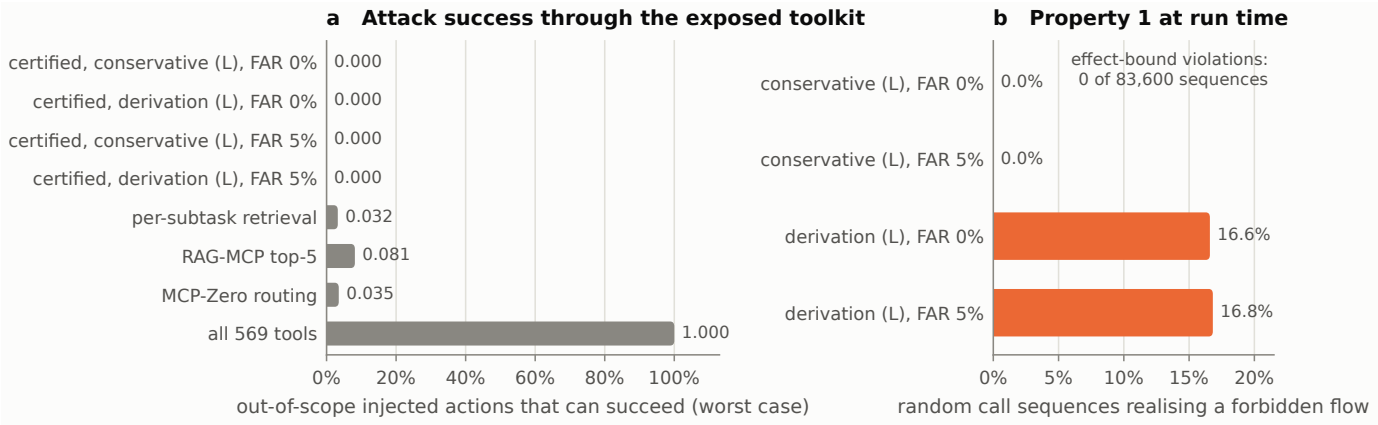


Fig. 10. (a) Worst-case success of injected actions outside the task's needed effect classes through the exposed toolkit; 2,136 task–seed–attack instances per system. (b) Share of random call sequences over certified toolkits that realise a forbidden information flow, by how (L) is checked.

TABLE 10
Request-path ablations on the LiveMCPBench test split (95 tasks). Each row changes one component of the reference configuration; differences to the reference in parentheses. Abst.: share of tasks with an explicit abstention.

Variant	Recall	$\lvert K\rvert$	Abst.
Reference (dev-tuned)	.186	1.42	0.06
No relevance term (Eq. (6))	.082 (−0.104)	1.37 (−0.05)	0.06
No structural score	.173 (−0.013)	1.39 (−0.03)	0.07
No backward closure ($d = 0$)	.182 (−0.004)	1.37 (−0.05)	0.07
No refinement (Alg. 3A)	.182 (−0.004)	1.39 (−0.03)	0.08
Alg. 1 read literally	.175 (−0.011)	1.93 (+0.51)	0.09
Strict rule 2	.182 (−0.004)	1.92 (+0.49)	0.07
Rule 1 off	.193 (+0.007)	1.45 (+0.03)	0.05
Haiku ρ_{LLM} profile	.186 (+0.000)	1.42 (+0.00)	0.06
Haiku analyst	.176 (−0.010)	1.60 (+0.18)	0.04
(L) along derivations	.186 (+0.000)	1.42 (+0.00)	0.06

8.8 Scalability and Evolution (RQ4)

Bitmap eligibility over three properties takes 1.9, 2.4 and 5.4 μs for 10^3, 10^4 and 10^5 tools, against 1.5, 19.0 and 174.8 ms for a scan, with identical results. Over 600 random insertions, deletions and re-hashes against 16 cached sets, an entry was invalidated exactly when its set changed (3,615 changed and invalidated, 5,985 unchanged and kept, no stale answer), as Proposition 3 states, and Lemma 1 held in all 300 random visibility trials. On the 569-tool registry an incremental `list_changed` update takes 29.9 ms against 3.64 s for a rebuild. A definition changed after verification loses its observed evidence (strength 0.73 to 0.25) and regains it only when the verified definition returns. RQ4 is answered positively.

8.9 Contract Typing and Analyst Vocabulary

Execution benchmarks such as MCP-Bench [10] pass the discovery layer only each tool's name, description and input schema. The 525 LiveMCPTool tools were presented in that format with the 95 test tasks, and a task counts as skipped when the selection is an abstention or empty. With lexical typing and the documented default policy (read, egress and session effects), 92–99% of the tasks are skipped whatever the analyst. With LLM-assisted typing and an open policy, the analyst of the main experiments skips 9% (recall 0.181); an analyst prompt whose type examples lie outside Θ produced out-of-vocabulary types in all 95 specifications and skips 86%; aligning its vocabulary with Θ reduces this to 51%, and also passing it the registry's domains, as line 2 of Algorithm 1 prescribes, to 36%. Typing and a shared vocabulary are therefore load-bearing. Lexically derived effects also make restrictive policies over-block: on the 57 tasks whose gold tools are all read-only under J, the default policy skips 35% (11% under the open policy), 13 of these 20 skips being χ_2 effect-bound counterexamples.

8.10 Threats to Validity

Construct validity. Exposure and attack success are measured against an LLM-labelled J and a simulated verifier: they establish soundness relative to J, not that real servers behave as labelled. Gold-tool recall is a proxy for task success that penalises inclusion-minimal toolkits when a gold chain lists alternative tools for one goal; no task was executed end to end. *Internal validity.* Parameters were tuned on the development split only, and the trust policy and cost weights were fixed in advance; analyst outputs are one sample per prompt from one model family. *External validity.* The registries hold 569 tools; the scalability results use generated contracts, the dense retriever is a stand-in, and timings come from a single machine.

REFERENCES

[1] G. Mo, W. Zhong, J. Chen, Q. Yuan, X. Chen, Y. Lu, H. Lin, B. He, X. Han, and L. Sun, "LiveMCPBench: Can agents navigate an ocean of MCP tools?" *arXiv preprint arXiv:2508.01780*, 2025.

[2] S. Fan, X. Ding, L. Zhang, and L. Mo, "MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark," *arXiv preprint arXiv:2508.07575*, 2025.

[3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[4] T. Gan and Q. Sun, "RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation," *arXiv preprint arXiv:2505.03275*, 2025.

[5] X. Fei, X. Zheng, and H. Feng, "MCP-Zero: Active tool discovery for autonomous LLM agents," *arXiv preprint arXiv:2506.01056*, 2025.

[6] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," *arXiv preprint arXiv:2508.14925*, 2025.

[7] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[8] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.

[9] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," *arXiv preprint arXiv:2504.11703*, 2025.

[10] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv preprint arXiv:2508.20453*, 2025.